

Examen réparti 2 DataCloud Master 2 SAR

Jonathan Lejeune

Février 2023

2 heures – **Tout document papier interdit**

Tout appareil électronique interdit

- Le barème est sur 21 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Lors de la correction il sera tenu compte de la simplicité et de la lisibilité de vos réponses
- S'abstenir de répondre est préférable à répondre une absurdité
- Vous trouverez en annexe de la documentation ou des rappels sur les API manipulées

Exercice 1 – Blockchain (4 points)

Merci de répondre directement sur le polycopié dédié à cette partie. N'oubliez pas d'y indiquer votre nom et votre prénom.

Exercice 2 – Questions de cours (6 points)

Cet exercice est fait pour évaluer la compréhension des différents concepts et mécanismes vus en cours. **Soyez concis et synthétique.**

Question 1 1 point

En fonction de quel(s) critère(s), le leader de zookeeper est-il élu ? Pourquoi élire le leader en fonction de ce(s) critère(s) est-il judicieux ?

Question 2 1 point

Dans Cassandra qu'est-ce qu'un SSTable ?

Question 3 1 point

Dans Cassandra qu'est ce que le flushing et quand est-il déclenché ?

Question 4 1 point

Dans Cassandra comment se passe une écriture en local sur un nœud ?

Question 5 1 point

Citez 6 critères parmi les 8 vus en cours qui peuvent caractériser un système distribué.

Question 6 1 point

Dans un système distribué qu'est ce que l'élasticité d'une infrastructure ? Quelle différence y a-t-il entre l'élasticité verticale et l'élasticité horizontale ?

Exercice 3 – Virtualisation de ressources (11 points)

L'entreprise Mississippi offre un service de location de ressources de calcul à ses clients grâce à des

techniques de virtualisation. Ce service consiste à allouer et à déployer des machines virtuelles (dénové VM par la suite) sur une infrastructure physique. Chaque machine virtuelle (et les ressources qui lui sont allouées) sont ainsi à usage exclusif d'un client donné. L'infrastructure est un ensemble de machines physiques (dénové PM par la suite) reliées par un réseau à haut débit. La facturation au client pour une VM se fait en fonction de deux éléments :

- du temps en heure où la VM est déployée et opérationnelle sur l'infrastructure. Le tarif horaire dépend de la configuration de la VM (plus la VM a de ressource plus le montant est important)
- de la quantité de données en octets envoyée et reçue via le réseau. Le tarif unitaire de l'octet envoyé ou reçu est le même pour tous les clients.

Pour payer, les clients créditent un compte qui est débité en fonction de la consommation de VM qu'ils utilisent.

Question 1 1/2 point

À quel type de modèle de système distribué ceci se réfère-t-il ?

Le système d'information gérant ce service est déployé sur une infrastructure dédiée. Il utilise deux SGBD NoSQL : Cassandra et Hbase.

Cassandra stocke les données relatives :

- aux clients
- aux machines physiques hébergeant les VMs
- aux différentes configurations de VM offertes aux clients

Les commandes CQL qui ont été lancées pour générer le schéma de données dans Cassandra sont les suivantes

```

1 CREATE KEYSPACE IF NOT EXISTS vmservice WITH REPLICATION = { 'class' : 'SimpleStrategy
  ', 'replication_factor': 3 };
2
3 USE vmservice;
4
5 CREATE TABLE Client (login VARCHAR, nom VARCHAR, prenom VARCHAR, adresse VARCHAR,
  email VARCHAR, credit DOUBLE, PRIMARY KEY (login));
6
7 CREATE TABLE PM(adresse_mac VARCHAR, adresse_ip VARCHAR, nb_cpu INT, taille_memoire
  INT, taille_disque INT, PRIMARY KEY (adresse_mac));
8
9 CREATE TABLE Config (nom VARCHAR, tarif_horaire DOUBLE, ,nb_cpu INT, taille_memoire
  INT, taille_disque INT, PRIMARY KEY (nom));
  
```

Hbase est utilisé pour maintenir les informations sur l'état courant d'utilisation des ressources de calcul. Ainsi, il existe une table Hbase "VM" dans le namespace "vmservice" qui est organisé de la manière suivante :

- la rowkey désigne un identifiant de VM
- une famille de colonne "info" contenant les colonnes suivantes :
 - ◊ "PM" indiquant l'adresse mac de la PM sur laquelle la VM est actuellement hébergée
 - ◊ "client" indiquant le login du client à qui appartient la VM
 - ◊ "config" indiquant le nom de la configuration de la VM
- une famille de colonne "conso" contenant les colonnes qui maintiennent les données de consommation de la VM, à savoir :
 - ◊ "utime" indiquant le temps en heure où la VM a été opérationnelle
 - ◊ "datanet" indiquant en octet la quantité de données émises et reçues par la VM sur le réseau

On supposera que les rowkeys et toutes les données de "info" sont stockées en tableau de carac-

tères ASCII et que les données de "conso" sont stockées sous le format d'un entier de 64 bits. Les données de consommation sont mises à jour régulièrement dans la table "VM" par des outils de monitoring.

Question 2 1/2 point

On suppose que la table VM possède 50 000 lignes et qu'elle est découpée en 10 régions. Quel est le coût minimal de stockage en termes de fichiers HDFS? **Justifier**

Question 3 2 points

Chaque PM exécute un hyperviseur pour leur virtualisation. Chaque hyperviseur monitor la quantité de données entrantes et sortantes des VM qu'il gère ainsi que leur temps où elles sont opérationnelles. Ils envoient périodiquement ces données vers un service spark streaming qui met à jour les données de consommation de la table Hbase. Cette mise à jour se fait par la méthode `update(ds:DStream[(String,Long,Long)]):Unit` qui prend en paramètre un flux de triplet composé de l'identifiant de la VM, de son temps d'opérationnalité et de sa quantité de données réseau reçues/envoyées depuis la dernière mesure. Donner le code de cette méthode (on supposera que les nœuds zookeeper sont les machines *z1*, *z2* et *z3*).

Nous définissons les classes scala suivantes :

```

1
2  case class VM(id:String,PM:String,client:String,config:String,utime:Long,datanet:
   Long)
3
4  case class Client(login:String,nom:String,prenom:String,adresse:String,mail:String,
   credit:Double)
5
6  case class Config(nom:String,tarif:Double,nb_cpu:Int,taille_memoire:Int,
   taille_disque:Int)
7
8  case class PM(mac:String,ip:String,nb_cpu:Int,taille_memoire:Int,taille_disque:Int)

```

Question 4 1 point

Donner le code d'une méthode `getClients(sc:SparkContext):RDD[Client]` qui pour un contexte spark donné et supposé correctement paramétré pour se connecter à Cassandra renvoie l'ensemble des clients présents dans la table "Client".

On supposera par la suite qu'il existe également les méthodes `getPMs(sc:SparkContext):RDD[PM]` et `getConfigs(sc:SparkContext):RDD[Config]` mais dont le code n'est pas demandé.

Question 5 1 1/2 points

Définir une fonction `getColumnns` qui permet de charger l'ensemble d'une table hbase dans un RDD (indépendamment de la table VM). Cette fonction prend en paramètre :

- un contexte spark
- un quorum zookeeper (String)
- le nom d'une table (String)
- un tableau de triplets qui décrit les colonnes que l'on souhaite récupérer et traduire en objet scala :
 - ◇ le premier élément est le nom de la famille de colonne (String)
 - ◇ le deuxième élément le nom de la colonne (String)
 - ◇ le troisième élément est une fonction de conversion d'un tableau d'octets à un Any

La fonction renvoie un RDD de couples dont l'élément de gauche est la rowkey sous format **String** et l'élément de droite est un **Array[Any]** dont l'élément *i* correspond à la valeur de la *i*ème colonne renseignée dans le tableau de triplets passé en paramètre.

Question 6 1½ points

Écrire la fonction `getVMs(sc:SparkContext):RDD[VM]`. Elle appelle la méthode `getColumns` définie précédemment et convertit chaque élément en VM. Pour rappel le type **Any** offre la méthode `def asInstanceOf[T]: T` qui permet de convertir un **Any** en **T**. On supposera que les nœuds zookeeper sont les machines *z1*, *z2* et *z3*.

Soit la fonction `req` suivante :

```

1  val NETPRICE=0.05 // tarif unitaire d'utilisation du réseau
2
3  def req(sc:SparkContext)={
4      val configs = getConfigs(sc).map(c=> (c.nom,c.tarif))
5      val clients = getClients(sc).map(c=> (c.login,c.credit))
6      val vms = getVMs(sc)
7      val rdd= vms.map(vm => (vm.config,(vm.client, vm.utime, vm.datanet*NETPRICE)))
8      val rdd1 = rdd.join(configs)
9      val rdd2 = rdd1.map(_._2)
10     val rdd3 = rdd2.map(c=> ( c._1._1, (c._1._2 * c._2 + c._1._3 ) ) )
11     val rdd4 = rdd3.reduceByKey(_+_ )
12     val rdd5 = rdd4.join(clients).mapValues(c=> c._2-c._1)
13     rdd5.filter(_._2 < 0.0)
14 }

```

Question 7 1 point

Que permet de calculer cette fonction ? Quel est son type de retour ?

Question 8 3 points

Proposer une modification de l'organisation des données stockées dans Hbase pour optimiser un maximum cette requête tout en ayant la possibilité d'obtenir facilement l'ensemble des VM existantes. Justifiez vos choix en donnant les changements qu'il faudrait faire sur le code des méthodes `req` et `getVMs`.

Annexes

```

1  object SparkHbaseConnector {
2      class MySparkContext(sc:SparkContext){
3          def hbaseTableRDD(tablename:String, scan:Scan, quorumzookeeper:String="localhost"
4              ):RDD[(ImmutableBytesWritable,Result)]={
5              val hbaseconf = HBaseConfiguration.create()
6              hbaseconf.set("hbase.zookeeper.quorum", quorumzookeeper);
7              hbaseconf.set(TableInputFormat.INPUT_TABLE, tablename);
8              hbaseconf.set(TableInputFormat.SCAN, TableMapReduceUtil.convertScanToString(
9                  scan));
10             return sc.newAPIHadoopRDD(hbaseconf, classOf[TableInputFormat], classOf[
11                 ImmutableBytesWritable], classOf[Result]);
12         }
13     }
14     implicit def sparkContextToMySparkContext(sc:SparkContext):MySparkContext= new
15         MySparkContext(sc)
16
17
18     class RDDHbase[M<:Mutation](rdd:RDD[M]){
19         def saveAsHbaseTable(tablename:String, quorumzookeeper:String="localhost"):Unit={
20             rdd.foreachPartition( it => {
21                 val hbaseconf = HBaseConfiguration.create()
22                 hbaseconf.set("hbase.zookeeper.quorum", quorumzookeeper)
23                 val connection = ConnectionFactory.createConnection(hbaseconf)
24                 val table = connection.getTable(TableName.valueOf(tablename))
25                 val puts = new ArrayList[Put]
26                 val deletes = new ArrayList[Delete]
27                 for(mut <- it){
28                     mut match {
29                         case put:Put => puts.add(put)
30                         case delete:Delete => deletes.add(delete)
31                         case append:Append => table.append(append)
32                         case increment:Increment => table.increment(increment)
33                         case _ => throw new RuntimeException("Bad type of mutation")
34                     }
35                 }
36                 if(!puts.isEmpty){
37                     table.put(puts)
38                 }
39                 if(!deletes.isEmpty()){
40                     table.delete(deletes)
41                 }
42                 table.close()
43                 connection.close()
44             })
45         }
46     }
47     implicit def rddToRDDHbase[M<:Mutation](rdd:RDD[M]): RDDHbase[M] = new RDDHbase(rdd)
48 }

```

Rappel des principales méthodes de l'API Java de Hbase

Connection

- Admin getAdmin()
- Table getTable(TableName tableName)

Table

- TableName getName()
- **boolean** exists(Get get)
- Result get(Get get)
- ResultScanner getScanner(Scan scan)//ResultScanner is an iterable of Result objects
- **void** put(Put put)
- **void** put(List<Put> puts)
- **void** delete(Delete delete)
- **void** delete(List<Delete> deletes)
- Result increment(Increment increment)//returns values of columns after the increment

TableName

- **static** TableName valueOf(String namespace, String tablequalifier)
- String getQualifierAsString()
- String getNamespaceAsString()
- String getNameAsString()//namespace:qualifier

Mutation (abstract)

- **byte** [] getRow()

Put

- Put(**byte**[] rowkey)
- Put(**byte**[] rowkey, **long** def_ts)
- Put addColumn(**byte**[] family, **byte**[] col, **byte**[] val);
- Put addColumn(**byte**[] family, **byte**[] col, **long** ts, **byte**[] val);

Delete

- Delete(**byte**[] rowkey)
- Delete(**byte**[] rowkey, **long** def_ts)
- Delete addColumns(**byte** [] family, **byte** [] qualifier);
- Delete addColumns(**byte** [] family, **byte** [] qualifier, **final long** timestamp);

Append

- Append(**byte**[] row)
- add(**byte** [] family, **byte** [] qualifier, **byte** [] value))

Increment

- Increment(**byte**[] row)
- Increment addColumn(**byte**[] family, **byte**[] qualifier, **long** amount)

Scan

- Scan()//scan all the table
- Scan(**byte**[] startRow, **byte**[] stopRow)
- Scan addFamily(**byte** [] family)
- Scan addColumn(**byte** [] family, **byte** [] qualifier)
- **byte**[] getStartRow()
- **byte**[] getStopRow()

Get

- Get(**byte** [] row)
- Get addFamily(**byte** [] family)
- Get addColumn(**byte** [] family, **byte** [] qualifier)

Result

- **byte**[] getValue(**byte**[] family, **byte**[] qualifier)
- Map<**byte**[], **byte**[]> getFamilyMap(**byte**[] family)

API for SparkContext	Meaning
new SparkContext (config: SparkConf)	Create a SparkContext that loads settings from a Spark Config object describing the application configuration. Any settings in this config overrides the default configs as well as system properties.
new SparkContext (master: String, appName: String, conf: SparkConf)	Alternative constructor that allows setting common Spark properties directly, : Cluster URL to connect to (e.g. mesos://host:port, spark://host:port, local[4]), a name for your application, to display on the cluster web UI and a SparkConf object specifying other Spark parameters
textFile (path: String, minPartitions: Int = defaultMinPartitions): RDD[String]	Read a text file from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI, and return it as an RDD of Strings. Path is the path to the text file on a supported file system, minPartitions is the suggested minimum number of partitions for the resulting RDD. It returns RDD of lines of the text file
sequenceFile [K, V](path: String, keyClass: Class[K], valueClass: Class[V]): RDD[(K, V)]	Get an RDD for a Hadoop SequenceFile with given key and value types. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, keyClass (resp. valueClass) is the Class of the key (resp. the value) associated with SequenceFileInputFormat. It returns RDD of tuples of key and corresponding value.
parallelize [T](seq: Seq[T], numSlices: Int): RDD[T]	Distribute a local Scala collection to form an RDD. NumSlices is number of partitions to divide the collection into. It returns the RDD representing distributed collection
newAPIHadoopFile [K, V, F <: InputFormat[K, V]](path: String, fClass: Class[F], kClass: Class[K], vClass: Class[V], conf: Configuration = hadoopConfiguration): RDD[(K, V)]	Get an RDD for a given Hadoop file with an arbitrary new API InputFormat and extra configuration options to pass to the input format. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, fclass is the storage format of the data to be read , kClass (resp. vClass) is the Class of the key (resp. the value) Hadoop configuration. It returns RDD of tuples of key and corresponding value
broadcast [T](value: T): Broadcast[T]	Broadcast a read-only variable to the cluster, returning a org.apache.spark.broadcast.Broadcast object for reading it in distributed functions. The variable will be sent to each cluster only once.
collectionAccumulator [T]: CollectionAccumulator[T]	Create and register a CollectionAccumulator, which starts with empty list and accumulates inputs by adding them into the list.
doubleAccumulator : DoubleAccumulator longAccumulator : LongAccumulator	Create and register a double or long accumulator, which starts with 0 and accumulates inputs by add.
register (acc: AccumulatorV2[_, _]): Unit	Register the given accumulator. Accumulators must be registered before use, or it will throw exception.
stop (): Unit	Shut down the SparkContext.

Simple transformation for RDD[T]	Meaning
map [U](f: (T) =>U): RDD[U]	Return a new distributed dataset formed by passing each element of the source through a function f .
mapValues [U](f: V => U): RDD[(K, U)]	Pass each value in the key-value pair RDD through a map function without changing the keys this also retains the original RDD's partitioning.
filter (f: (T) =>Boolean): RDD[T]	Return a new dataset formed by selecting those elements of the source on which f returns true.
flatMap [U](f: (T) =>TraversableOnce[U]): RDD[U]	Similar to map, but each input item can be mapped to 0 or more output items (so f should return a Seq rather than a single item).
flatMapValues [U](f: V => TraversableOnce[U]): RDD[(K, U)]	Pass each value in the key-value pair RDD through a flatMap function without changing the keys ; this also retains the original RDD's partitioning.

union (other: RDD[T]): RDD[T]	Return a new dataset that contains the union of the elements in the source dataset and the argument.
zip [U](other: RDD[U]): RDD[(T, U)]	Zips this RDD with another one, returning key-value pairs with the first element in each RDD, second element in each RDD, etc. <i>Assumes that the two RDDs have the same number of partitions and the same number of elements in each corresponding partition</i> (e.g. one was made through a map on the other).
<u>zipWithIndex</u> (): RDD[(T, Long)]	Zips this RDD with its element indices. The ordering is first based on the partition index and then the ordering of items within each partition. So the first item in the first partition gets index 0, and the last item in the last partition receives the largest index. This is similar to Scala's zipWithIndex but it uses Long instead of Int as the index type. This method needs to trigger a spark job when this RDD contains more than one partitions.
keyBy [K](f: (T) =>K): RDD[(K, T)]	Creates tuples of the elements in this RDD by applying f
glom (): RDD[Array[T]]	Return an RDD created by coalescing all elements within each partition into an array.

Large transformation for RDD[T]	Meaning
intersection (other: RDD[T]): RDD[T] intersection (other: RDD[T], numPartitions: Int): RDD[T] intersection (other: RDD[T], part: Partitioner): RDD[T]	Return the intersection of this RDD and another one. The output will not contain any duplicate elements, even if the input RDDs did.
distinct (): RDD[T] distinct (numPartitions: Int): RDD[T]	Return a new dataset that contains the distinct elements of the source dataset.
cartesian [U](other: RDD[U]): RDD[(T, U)]	When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).
repartition (numPartitions: Int): RDD[T]	Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.
subtract (other: RDD[T]): RDD[T] subtract (other: RDD[T], numPartitions: Int): RDD[T] subtract (other: RDD[T], p: Partitioner): RDD[T]	Return an RDD with the elements from <i>this</i> that are not in <i>other</i> .
sortBy [K](f: (T) =>K, ascending: Boolean = true, numPartitions: Int = this.partitions.length): RDD[T]	Return this RDD sorted by the given key function.

Large transformation for RDD[T] and T= (K,V)	Meaning
groupByKey (): RDD[(K, Iterable[V])] groupByKey (numPartitions:Int): RDD[(K, Iterable[V])] groupByKey (p:Partitioner): RDD[(K, Iterable[V])]	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs.
reduceByKey (func: (V, V) =>V): RDD[(K, V)] reduceByKey (numPartitions:Int, func: (V, V) =>V): RDD[(K, V)] reduceByKey (partitioner: Partitioner, func: (V, V) =>V): RDD[(K, V)]	When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function <i>func</i> , which must be of type (V,V) => V.
sortByKey (ascending: Boolean = true, numPartitions: Int = self.partitions.length) : RDD[(K, V)]	When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean <i>ascending</i> argument.

join [W](other: RDD[(K, W)]): RDD[(K, (V, W))] join [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, W))] join [W](other: RDD[(K, W)], p: Partitioner): RDD[(K, (V, W))]	Return an RDD containing all pairs of elements with matching keys in <i>this</i> and <i>other</i> . Each pair of elements will be returned as a (k, (v1, v2)) tuple, where (k, v1) is in <i>this</i> and (k, v2) is in <i>other</i>
leftOuterJoin [W](other: RDD[(K, W)]): RDD[(K, (V, Option[W]))] leftOuterJoin [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, Option[W]))] leftOuterJoin [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (V, Option[W]))]	Perform a left outer join of <i>this</i> and <i>other</i> . For each element (k, v) in <i>this</i> , the resulting RDD will either contain all pairs (k, (v, Some(w))) for w in <i>other</i> , or the pair (k, (v, None)) if no elements in <i>other</i> have key k.
rightOuterJoin [W](other: RDD[(K, W)]): RDD[(K, (Option[V], W))] rightOuterJoin [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], W))] rightOuterJoin [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], W))]	Perform a right outer join of <i>this</i> and <i>other</i> . For each element (k, w) in <i>other</i> , the resulting RDD will either contain all pairs (k, (Some(v), w)) for v in <i>this</i> , or the pair (k, (None, w)) if no elements in <i>this</i> have key k.
fullOuterJoin [W](other: RDD[(K, W)]): RDD[(K, (Option[V], Option[W]))] fullOuterJoin [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], Option[W]))] fullOuterJoin [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], Option[W]))]	Perform a full outer join of <i>this</i> and <i>other</i> . For each element (k, v) in <i>this</i> , the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for w in <i>other</i> , or the pair (k, (Some(v), None)) if no elements in <i>other</i> have key k. Similarly, for each element (k, w) in <i>other</i> , the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for v in <i>this</i> , or the pair (k, (None, Some(w))) if no elements in <i>this</i> have key k.
cogroup [W](o: RDD[(K, W)]): RDD[(K, (Iterable[V], Iterable[W]))] cogroup [W](o: RDD[(K, W)], numPart: Int): RDD[(K, (Iterable[V], Iterable[W]))] cogroup [W](o: RDD[(K, W)], p: Partitioner): RDD[(K, (Iterable[V], Iterable[W]))]	When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called groupWith .
subtractByKey [W](other: RDD[(K, W)]): RDD[(K, V)] subtractByKey [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, V)] subtractByKey [W](other: RDD[(K, W)], p: Partitioner): RDD[(K, V)]	Return an RDD with the pairs from <i>this</i> whose keys are not in <i>other</i>

Action for RDD[T]	Meaning
reduce (f: (T, T) =>T): T	Aggregate the elements of the dataset using a function <i>func</i> (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
collect (): Array[T]	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
count ():Long	Return the number of elements in the dataset.
first (): T	Return the first element of the dataset (similar to take(1)).
take (num: Int): Array[T]	Return an array with the first <i>n</i> elements of the dataset.
top (num: Int)(implicit ord: Ordering[T]): Array[T]	Returns the top k (largest) elements from this RDD as defined by the specified implicit Ordering[T] and maintains the ordering.
saveAsObjectFile (path: String): Unit	Save this RDD as a SequenceFile of serialized objects.
saveAsTextFile (path: String): Unit	Save this RDD as a text file, using string representations of elements.
foreach (f: (T) =>Unit): Unit	Run a function <i>func</i> on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems.

API for StreamingContext	Meaning
new StreamingContext (sparkContext: SparkContext, batchDuration: Duration)	Create a StreamingContext using an existing SparkContext. <i>BatchDuration</i> is the time interval at which streaming data will be divided into batches. Duration can be initialized by <i>Milliseconds(nb)</i> , <i>Seconds(nb)</i> or <i>Minutes(nb)</i>
checkpoint (directory: String)	Set the context to periodically checkpoint the DStream operations for driver fault-tolerance. The directory is HDFS-compatible directory where the checkpoint data will be reliably stored. Note that this must be a fault-tolerant file system like HDFS.
socketTextStream (hostname: String, port: Int, sl: StorageLevel = StorageLevel.MEMORY_AND_DISK_SER_2): ReceiverInputDStream[String]	Creates an input stream from TCP source hostname:port. Data is received using a TCP socket and the receive bytes is interpreted as UTF8 encoded <code>\n</code> delimited lines. The <i>hostname</i> parameter is the host to connect to for receiving data, <i>port</i> is the port to connect to for receiving data, <i>sl</i> is the Storage level to use for storing the received objects.
socketStream [T: ClassTag](hostname: String, port: Int, converter: (InputStream) => Iterator[T], sl: StorageLevel): ReceiverInputDStream[T]	Create an input stream from network source hostname:port, where data is received as serialized blocks (serialized using the Spark's serializer) that can be directly pushed into the block manager without deserializing them. This is the most efficient way to receive data. The <i>hostname</i> parameter is the host to connect to for receiving data, <i>port</i> is the port to connect to for receiving data, <i>sl</i> is the Storage level to use for storing the received objects. T is the type of the objects in the received blocks.
textFileStream (directory: String): DStream[String]	Create an input stream that monitors a Hadoop-compatible filesystem for new files and reads them as text files (using key as LongWritable, value as Text and input format as TextInputFormat). Files must be written to the monitored directory by "moving" them from another location within the same file system. File names starting with . are ignored.
start () : Unit	Start the execution of the streams. Throws IllegalStateException if the StreamingContext is already stopped.
awaitTermination ()	Wait for the execution to stop. Any exceptions that occurs during the execution will be thrown in this thread.
awaitTerminationOrTimeout (timeout: Long): Boolean	Wait for the execution to stop. Any exceptions that occurs during the execution will be thrown in this thread. The <i>timeout</i> parameter is the time to wait in milliseconds. Returns true if it's stopped; or throw the reported error during the execution; or false if the waiting time elapsed before returning from the method.
stop (stopSparkContext: Boolean = conf.getBoolean("spark.streaming.stopSparkContextByDefault", true)): Unit	Stop the execution of the streams immediately (does not wait for all received data to be processed). By default, if <i>stopSparkContext</i> is not specified, the underlying SparkContext will also be stopped. This implicit behavior can be configured using the SparkConf configuration <i>spark.streaming.stopSparkContextByDefault</i> . If <i>stopSparkContext</i> is true, it stops the associated SparkContext. The underlying SparkContext will be stopped regardless of whether this StreamingContext has been started.

Transformation for DStream[T]	Meaning
filter (filterFunc: (T) => Boolean): DStream[T]	Return a new DStream containing only the elements that satisfy a predicate.
map [U](mapFunc: (T) => U): DStream[U]	Return a new DStream by applying a function to all elements of this DStream
flatMap [U](flatMapFunc: (T) => TraversableOnce[U]): DStream[U]	Return a new DStream by applying a function to all elements of this DStream, and then flattening the results
glom (): DStream[Array[T]]	Return a new DStream in which each RDD is generated by applying glom() to each RDD of this DStream. Applying glom() to an RDD coalesces all elements within each partition into an array.
union (that: Dstream[T]): DStream[T]	Return a new DStream by unifying data of another DStream with this DStream.
repartition (numPartitions: Int): DStream[T]	Return a new DStream with an increased or decreased level of parallelism. Each RDD in the returned DStream has exactly numPartitions partitions.
count (): DStream[Long]	Return a new DStream in which each RDD has a single element generated by counting each RDD of this DStream.
countByValue (numPartitions: Int = ssc.sc.defaultParallelism)(implicit ord: Ordering[T] = null): DStream[(T, Long)]	Return a new DStream in which each RDD contains the counts of each distinct value in each RDD of this DStream. Hash partitioning is used to generate the RDDs with numPartitions partitions (Spark's default number of partitions if numPartitions not specified).
countByValueAndWindow (windowDuration: Duration, slideDuration: Duration, numPartitions: Int = ssc.sc.defaultParallelism)(implicit ord: Ordering[T] = null): DStream[(T, Long)]	Return a new DStream in which each RDD contains the count of distinct elements in RDDs in a sliding window over this DStream. Hash partitioning is used to generate the RDDs with numPartitions partitions (Spark's default number of partitions if numPartitions not specified).
transform [U](transformFunc: (RDD[T], Time) => RDD[U])(implicit arg0: ClassTag[U]): DStream[U]	Return a new DStream in which each RDD is generated by applying a function on each RDD of 'this' DStream.
reduce (reduceFunc: (T, T) => T): DStream[T]	Return a new DStream in which each RDD has a single element generated by reducing each RDD of this DStream.
foreachRDD (foreachFunc: (RDD[T]) => Unit): Unit	Apply a function to each RDD in this DStream. This is an output operator, so 'this' DStream will be registered as an output stream and therefore materialized.
countByWindow (windowDuration: Duration, slideDuration: Duration): DStream[Long]	Return a new DStream in which each RDD has a single element generated by counting the number of elements in a sliding window over this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions
reduceByWindow (reduceFunc: (T, T) => T, windowDuration: Duration, slideDuration: Duration): DStream[T]	Return a new DStream in which each RDD has a single element generated by reducing all elements in a sliding window over this DStream.
window (windowDuration: Duration, slideDuration: Duration): DStream[T]	Return a new DStream in which each RDD contains all the elements in seen in a sliding window of time over this DStream.

Transformation for DStream[T], T= (K,V)	Meaning
combineByKey [C](createCombiner: (V) => C, mergeValue: (C, V) => C, mergeCombiner: (C, C) => C, partitioner: Partitioner, mapSideCombine: Boolean = true): DStream[(K, C)]	Combine elements of each key in DStream's RDDs using custom functions. This is similar to the combineByKey for RDDs. Please refer to combineByKey in org.apache.spark.rdd.PairRDDFunctions in the Spark core documentation for more information.
reduceByKey (reduceFunc: (V, V) => V): DStream[(K, V)]	Return a new DStream by applying reduceByKey to each RDD. The values for each key are merged using the associative and commutative reduce function. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
mapValues [U](mapValuesFunc: (V) => U): DStream[(K, U)]	Return a new DStream by applying a map function to the value of each key-value pairs in 'this' DStream without changing the key
flatMapValues [U](flatMapValuesFunc: (V) => TraversableOnce[U]): DStream[(K, U)]	Return a new DStream by applying a flatmap function to the value of each key-value pairs in 'this' DStream without changing the key.
join [W](other: DStream[(K, W)]): DStream[(K, (V, W))]	Return a new DStream by applying 'join' between RDDs of this DStream and other DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
leftOuterJoin [W](other: DStream[(K, W)]): DStream[(K, (V, Option[W]))]	Return a new DStream by applying 'left outer join' between RDDs of this DStream and other DStream.
fullOuterJoin [W](other: DStream[(K, W)]): DStream[(K, (Option[V], Option[W]))]	Return a new DStream by applying 'full outer join' between RDDs of this DStream and other DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
rightOuterJoin [W](other: DStream[(K, W)]): DStream[(K, (Option[V], W))]	Return a new DStream by applying 'right outer join' between RDDs of this DStream and other DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
cogroup [W](other: DStream[(K, W)])(implicit arg0: ClassTag[W]): DStream[(K, (Iterable[V], Iterable[W]))]	Return a new DStream by applying 'cogroup' between RDDs of this DStream and other DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
groupByKey (): DStream[(K, Iterable[V])]	Return a new DStream by applying groupByKey to each RDD. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
groupByKeyAndWindow (windowDuration: Duration): DStream[(K, Iterable[V])]	Return a new DStream by applying groupByKey over a sliding window. This is similar to DStream.groupByKey() but applies it over a sliding window. The new DStream generates RDDs with the same interval as this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
reduceByKeyAndWindow (reduceFunc: (V, V) => V, windowDuration: Duration): DStream[(K, V)]	Return a new DStream by applying reduceByKey over a sliding window on this DStream. Similar to DStream.reduceByKey(), but applies it over a sliding window. The new DStream generates RDDs with the same interval as this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.

reduceByKeyAndWindow (reduceFunc: (V, V) => V, invReduceFunc: (V, V) => V, windowDuration: Duration, slideDuration: Duration = self.slideDuration, numPartitions: Int = ssc.sc.defaultParallelism, filterFunc: ((K, V)) => Boolean = null): DStream[(K, V)]	Return a new DStream by applying incremental `reduceByKey` over a sliding window. The reduced value of over a new window is calculated using the old window's reduced value : 1. reduce the new values that entered the window (e.g., adding new counts) 2. "inverse reduce" the old values that left the window (e.g., subtracting old counts) This is more efficient than reduceByKeyAndWindow without "inverse reduce" function. However, it is applicable to only "invertible reduce functions". Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
updateStateByKey [S](updateFunc: (Seq[V], Option[S]) => Option[S])	Return a new "state" DStream where the state for each key is updated by applying the given function on the previous state of the key and the new values of each key. In every batch the updateFunc will be called for each state even if there are no new values. Hash partitioning is used to generate the RDDs with Spark's default number of partitions. If updateFunc function returns None, then corresponding state key-value pair will be eliminated.

Output Operation for DStream[T]	Meaning
foreachRDD (foreachFunc: (RDD[T]) => Unit): Unit	Apply a function to each RDD in this DStream. This is an output operator, so 'this' DStream will be registered as an output stream and therefore materialized.
print (num: Int): Unit	Print the first num elements of each RDD generated in this DStream. This is an output operator, so this DStream will be registered as an output stream and there materialized.
print (): Unit	Print the first ten elements of each RDD generated in this DStream. This is an output operator, so this DStream will be registered as an output stream and there materialized.
saveAsObjectFiles (prefix: String, suffix: String = ""): Unit	Save each RDD in this DStream as a Sequence file of serialized objects. The file name at each batch interval is generated based on prefix and suffix: "prefix-TIME_IN_MS.suffix".
saveAsTextFiles (prefix: String, suffix: String = ""): Unit	Save each RDD in this DStream as at text file, using string representation of elements.
saveAsNewAPIHadoopFiles [F <: OutputFormat[K, V]](prefix: String, suffix: String): Unit	Save each RDD in this DStream as a Hadoop file. The file name at each batch interval is generated based on prefix and suffix: "prefix-TIME_IN_MS.suffix".